

StoreOps AI Development Harness

Capstone submission deliverables,
Application Architecture and Demonstrated
evidence

AI-Native Tech Architect Programme

Build Track
Solution Architect – Venkatesh.Rangarajan2@cognizant.com

Submission deck
29 Sep 2026



Submission package at a glance

All four required components are represented in the repository

45%

Working harness

17%

Harness Design Brief

28%

Demonstration artefacts

+10%

Architecture journal
bonus

Verified application evidence

101

tests passed

93.22%

line coverage

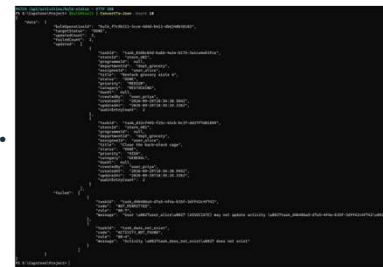
33/33

real-server smoke checks

0

blocking findings

Final evidence action: capture the healthy Docker container and bulk-status response in one screenshot.





StoreOps application

Retail operations API

Five domain modules support operational activities, programmes, staff coordination, alerts and performance reporting.

- **Activities** Tasks, Status, Priority, Audit trail
- **Programmes** Store initiatives and membership
- **Staff** Authentication and read-only identity ports
- **Alerts + Reports** Event-driven notifications and metrics

Node.js 20+, Express 4, strict TypeScript, Vitest

StoreOps Application architecture

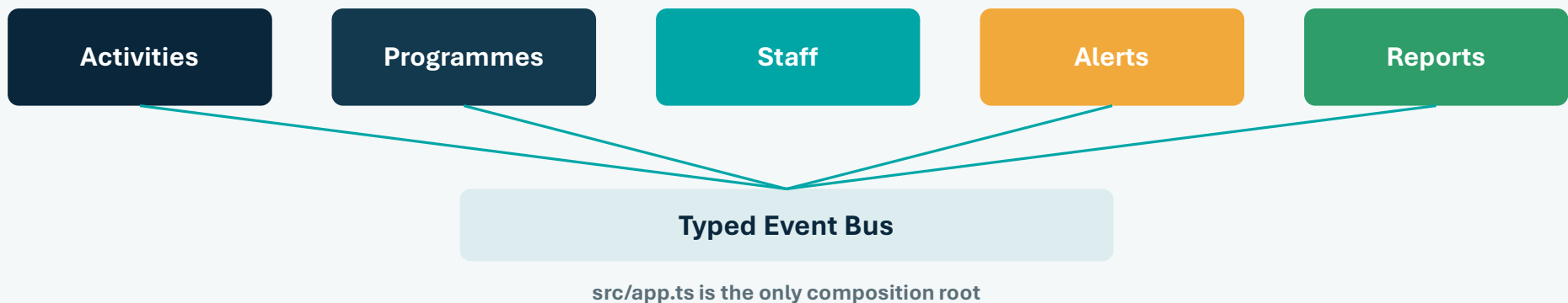
04

Layer ownership and event-driven cross-module effects



Cross-module reads use public read ports

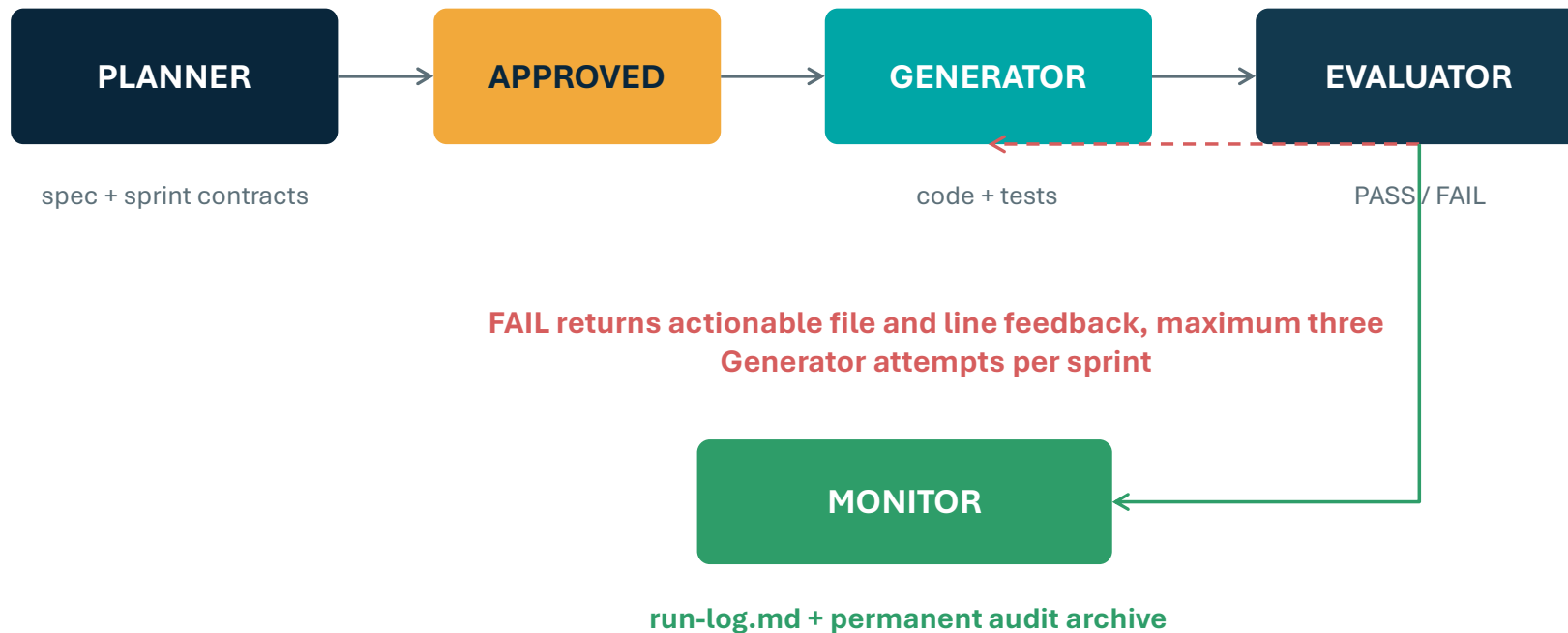
Cross-module writes publish typed events after persistence



Harness orchestration

05

The developer approves intent once; the governed loop handles implementation and review



File handoffs make every decision replayable and reviewable

Controls for the four observed failure modes

06

Each client concern maps to a deterministic check and a reviewer judgement

Risk	Observed failure	Deterministic control	Reviewer focus
F1	Sibling repository import	MB-* boundary rules	Ownership and read-port use
F2	Raw or hand-built errors	EH-* error rules	Correct AppError subtype
F3	Status-only tests	TQ-* rules + coverage	Test fails when rule is removed
F4	Missing event integration	EV-* catalogue rules	Payload sufficiency and consumers

Gate result

PASS

0 blocking findings

4 advisory event findings retained for trend monitoring

A blocking result cannot be averaged away or overruled by the Evaluator.

Demonstration feature: shift handover bulk update

07

PATCH /api/activities/bulk-status supports partial success with stable rule identifiers

REQUEST

**PATCH /api/activities/
bulk-status**

```
{  
  "taskIds": [  
    "task_1",  
    "task_2",  
    "task_chilled",  
    "task_missing"  
  ],  
  "targetStatus": "DONE"  
}
```

Evaluate BR-1...BR-9
before any write

Persist accepted subset
+ audit entry

Publish one completion
event after write

HTTP 200

updatedCount: 2

failedCount: 2

Updated

task_1, task_2

Failed

task_chilled NOT_PERMITTED / BR-7
task_missing
ACTIVITY_NOT_FOUND / BR-4

Alerts and reports react through their own subscribers

Code highlight: controlled write, then publish

The service keeps the transaction boundary inside activities and announces only committed changes

`src/modules/activities/activities.service.ts`

```
const tasks = await repository.findMany(request.taskIds);
const plan = planBulkStatusUpdate({ request, tasks, principal });

if (plan.rejections.length > 0) {
  throw new BusinessRuleError(primaryRule, message, options);
}

await repository.saveMany(updated);

await events.publish({
  type: 'activities.bulk_status.completed',
  correlationId,
  payload: { updated: plan.accepted, failedCount }
});
```

Why it matters

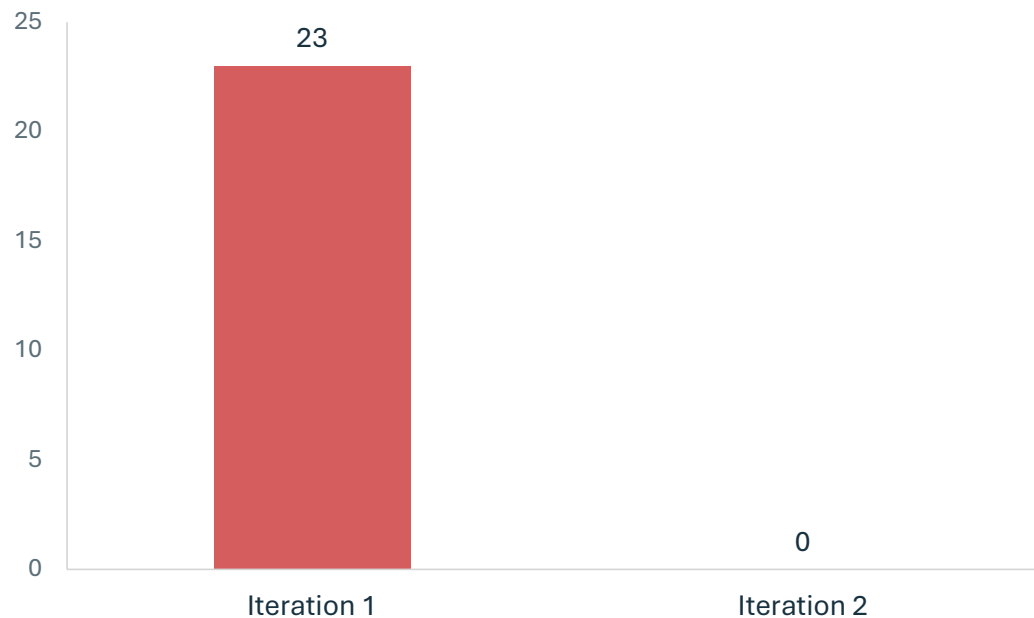
- 1 Pure rule planning**
Partial failure is decided before persistence.
- 2 Typed rejection**
BusinessRuleError carries the approved rule ID.
- 3 Owned write**
Activities writes only its own repository.
- 4 Event after commit**
Alerts and reports react without sibling writes.

The route validates input and delegates. It contains no business rules.

The Evaluator caught defects that tests missed

09

Sprint 2 iteration 1 compiled and passed its own tests but still contained 23 blocking findings



23 findings in iteration 1

F1 boundaries	3
F2 errors	5
F3 test quality	11
F4 events	4

Iteration 2

0 blocking
11/11 criteria MET

Green tests alone would have merged the rejected implementation.

Submission deliverables

10

Repository paths provide a direct route from rubric requirement to evidence

Component	Primary evidence	Status
Working harness	CLAUDE.md, agents, 8 canonical skills, checks	Complete
Design Brief	DESIGN_BRIEF.md, four required sections	Complete
Demonstration run	PROMPT.md + archived specs, contracts and verdicts	Complete
Deployment	Dockerfile, Compose, DEPLOYMENT.md, smoke evidence	Screenshot pending
Reflection	REFLECTION.md, 354 words	Complete
Architecture journal	JOURNAL.md	Bonus complete

Packaged submission [output/storeops-capstone-submission.zip](#)

184 files • source, harness, evidence and documents

Deployment evidence and final submission action

11

Application behaviour is verified; the Docker screenshot remains the final evidence item

VERIFIED REAL-SERVER RUN

```
> npm run smoke
```

```
GET /health          200 ok
PATCH /api/activities/bulk-status 200
updatedCount         2
failedCount           2
SHIFT_HANDOVER alert received
STORE_SUMMARY recompute ready
correlationId         preserved
```

```
SMOKE PASS: 33/33 checks
```

Docker evidence captured

- 1 **docker compose up --build -d**
- 2 **Create activities, then call the bulk endpoint**
- 3 **Capture one screenshot with healthy container and HTTP 200 response**

Saved in docs/evidence/docker-bulk-status.png